SMART CONTRACT SECURITY REVIEW

TSwap Audit Report

TSwap

Version 1.0

Joev
Independent Smart Contract Security Researcher

June 14, 2026

Table of Contents

About the Auditor

Joev is an independent smart contract security researcher.

- GitHub: https://github.com/joevly

Disclaimer

This document records a time-boxed security review performed by Joev on the specific code and commit identified in *Audit Details*. It describes the issues found within that window and is **not** a guarantee that the code is free of vulnerabilities — a clean report lowers risk, it does not remove it.

The review covers only the on-chain Solidity implementation in scope. Off-chain components, economic and incentive design, governance, deployment configuration, and third-party dependencies are out of scope unless explicitly stated. This review is not an endorsement of the protocol, its team, or its business, and nothing here is financial, investment, or legal advice. Ongoing testing, monitoring, and independent review remain the responsibility of the protocol team.

Protocol Summary

This project is meant to be a permissionless way for users to swap assets between each other at a fair price. You can think of T-Swap as a decentralized asset/token exchange (DEX). T-Swap is known as an Automated Market Maker (AMM) because it doesn't use a normal "order book" style exchange, instead it uses "Pools" of an asset. It is similar to Uniswap. To understand Uniswap, please watch this video: Uniswap Explained

Audit Details

The findings in this document correspond to the following commit hash:

```
e643a8d4c2c802490976b538dd009b351b1c8dda
```

Scope

```
./src/
#-- PoolFactory.sol
#-- TSwapPool.sol
```

- **Solc version:** 0.8.20
- **Chain(s) to deploy to:** Ethereum Mainnet.
- **Tokens:** Any ERC20 token.

Roles

- **Liquidity Providers:** Users who have liquidity deposited into the pools. Their shares are represented by the LP ERC20 tokens. They gain a 0.3% fee every time a swap is made.
- **Users:** Users who want to swap tokens.

Methodology & Tools

This review combined:

- **Manual review** — line-by-line reading of `PoolFactory.sol` and `TSwapPool.sol`, focused on the AMM accounting, fee math, slippage protection, and the core `x * y = k` invariant.
- **Dynamic testing** — Foundry unit tests and proof-of-concept exploits, plus fuzz/invariant testing to validate that swaps preserve the constant-product invariant.
- **Weird-ERC20 analysis** — reasoning about non-standard ERC20 behaviors and their effect on the pool accounting.

Risk Classification

Severity is a function of **likelihood** (how easily the issue can be triggered) and **impact** (how damaging it is if triggered):

		Impact		
		High	Medium	Low
Likelihood	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low

This Critical / High / Medium / Low model is the de-facto standard among audit firms and bug-bounty platforms (e.g. Cantina, OpenZeppelin, Immunefi). Informational and Gas findings sit outside the matrix as code-quality and optimization notes.

Severity Definitions

- **Critical** — directly and easily exploitable loss or theft of funds, or full protocol compromise. Must be fixed before deployment.
- **High** — loss of funds or a broken core invariant under realistic conditions; urgent fix.
- **Medium** — conditional or bounded harm, or protocol disruption where recovery is likely; fix recommended.
- **Low** — limited impact, unlikely preconditions, or a deviation from best practice with minor effect.
- **Informational** — no direct security impact: code quality, readability, unused code, events.
- **Gas** — optimizations that reduce gas with no change to behavior.

Executive Summary

The audit process took a week due to the many new vulnerabilities and hacking patterns I hadn't encountered before. I discovered over 12 vulnerabilities for this protocol. In this protocol, I saw and learned a lot vulnerabilities as Weird ERC-20, used Fuzzing test, and others.

Issues found

Severity	Number of issues found
Critical	0
High	4
Medium	1
Low	2
Info	5
Gas	0
Total	12

Findings Summary

ID	Short title	Severity	Status
H-1	Wrong fee scalar overcharges users	High	Open
H-2	No slippage protection in `swapExactOutput`	High	Open
H-3	`sellPoolTokens` swaps the wrong direction	High	Open
H-4	Swap bonus breaks the `x * y = k` invariant	High	Open
M-1	`deposit` ignores its `deadline` parameter	Medium	Open
L-1	`LiquidityAdded` event arguments out of order	Low	Open
L-2	`swapExactInput` always returns zero	Low	Open
I-1	Unused error in `PoolFactory`	Info	Open
I-2	Missing zero-address check in `PoolFactory`	Info	Open
I-3	Use `.symbol()` instead of `.name()`	Info	Open
I-4	`Swap` event is under-indexed	Info	Open
I-5	Missing zero-address check in `TSwapPool`	Info	Open

Findings

Critical

No critical severity issues found.

High

HIGH **[H-1] Incorrect fee calculation in `TSwapPool::getInputAmountBasedOnOutput` causes protocol to take too many tokens from users, resulting in lost fees**

Description: The `getInputAmountBasedOnOutput` function is intended to calculate the amount of tokens a user should deposit given an amount of tokens of output tokens. However, the function currently miscalculates the resulting amount. When calculating the fee, it scales the amount by 10_000 instead of 1_000.

Impact: Protocol takes more fees than expected from users.

Recommended Mitigation:

You should change the code to something like this:

```
    function getInputAmountBasedOnOutput(
        uint256 outputAmount,
        uint256 inputReserves,
        uint256 outputReserves
    )
        public
        pure
        revertIfZero(outputAmount)
        revertIfZero(outputReserves)
        returns (uint256 inputAmount)
    {
        return
-           ((inputReserves * outputAmount) * 10000) / ((outputReserves - outputAmount)
* 997);
+            ((inputReserves * outputAmount) * 1000) / ((outputReserves - outputAmount)
* 997);
    }
```

HIGH [H-2] Lack of slippage protection in `TSwapPool::swapExactOutput` causes users to potentially receive way fewer tokens

Description: The `SwapExactOutput` function does not include any sort by slippage protection. This function is similar to what is done in `TSwapPool::swapExactInput`, where the function specifies a `minOutputAmount`, the `swapExactOutput` function should specify a `maxInputAmount`.

Impact: If market conditions change before the transaction processes, the user could get a much worse swap.

Proof of Concept:

You should test with `test_swapExactOutput_hasNoMaxInputAmount_userOverpays()` function in `TSwapPool.t.sol`:

additional `whale` accounts in `setUp()` to help testing:

```
function setUp() public {
.
.
.
+       weth.mint(whale, 100e18);
+       poolToken.mint(whale, 100e18);
    }
```

After that, you can run the main test:

```
    function test_swapExactOutput_hasNoMaxInputAmount_userOverpays() public {
        // PURPOSE: prove that swapExactOutput has NO slippage protection.
        // It has no `maxInputAmount` parameter, so a user cannot cap how much
        // input they are willing to pay. If the price moves against them before
        // their tx executes (volatility / MEV front-run), they silently overpay
        // and the swap STILL succeeds instead of reverting.

        uint256 OUTPUT_AMOUNT = 5e18;     // user wants exactly 5 poolToken out
        uint256 INPUT_RESERVES = 100e18;  // calm-market WETH reserve (after deposit)
```

```
        uint256 OUTPUT_RESERVES = 100e18; // calm-market poolToken reserve (after
deposit)

        // Give the user enough WETH to cover the inflated price. setUp() already
        // mints 10e18; the volatile price costs ~222 WETH, so mint 220e18 more
        // (220 + 10 = 230e18). This guarantees the test fails because of the BUG,
        // not because of an insufficient balance.
        weth.mint(user, 220e18);

        // --- ARRANGE: LP seeds the pool with 100 WETH : 100 poolToken ---
        vm.startPrank(liquidityProvider);
        weth.approve(address(pool), INPUT_RESERVES);
        poolToken.approve(address(pool), OUTPUT_RESERVES);
        pool.deposit(INPUT_RESERVES, 90e18, OUTPUT_RESERVES, uint64(block.timestamp));
        vm.stopPrank();

        // Baseline "fair price": the input the user SHOULD pay for OUTPUT_AMOUNT
        // in the calm 100:100 pool. Captured BEFORE the whale moves the price.
        // NOTE: this value is itself ~10x too high due to a SEPARATE bug in
        // getInputAmountBasedOnOutput (magic number 10000 should be 1000), but
        // it still works as a relative baseline for this slippage proof.
        uint256 expectedNormal = pool.getInputAmountBasedOnOutput(OUTPUT_AMOUNT,
INPUT_RESERVES, OUTPUT_RESERVES);

        // --- ACT 1: the whale creates volatility ---
        // A large swap drains poolToken, making it scarce/expensive. This stands
        // in for a front-run / market move landing just before the user's tx.
        vm.startPrank(whale);
        weth.approve(address(pool), 100e18);
        pool.swapExactInput(weth, 100e18, poolToken, 40e18, uint64(block.timestamp));
        vm.stopPrank();

        // --- ACT 2: the victim swaps at the now-worse price ---
        vm.startPrank(user);
        uint256 startingUserBalance = weth.balanceOf(user);
        weth.approve(address(pool), type(uint256).max);
        pool.swapExactOutput(weth, poolToken, OUTPUT_AMOUNT, uint64(block.timestamp));
        uint256 endingUserBalance = weth.balanceOf(user);
        vm.stopPrank();

        // Actual WETH that left the user's wallet for the same 5 poolToken.
        uint256 wethUserPaid = startingUserBalance - endingUserBalance;

        // --- ASSERT: the user paid far more than the fair price, yet the swap
        // succeeded. With a maxInputAmount this call would have reverted. ---
        assert(wethUserPaid > expectedNormal * 2);

        console.log("Expected WETH User Paid (calm price): ", expectedNormal);
        // 52.789948793749670062 WETH
        console.log("Actual WETH User Paid (after volatility): ", wethUserPaid);
        // 222.519471985918317079 WETH  -> ~4.2x the calm price, with NO revert
    }
```

Recommended Mitigation:

We should include a `maxInputAmount` so the user only has to spend up to a spesific amount and can predict how much they will spend on the protocol.

```
    function swapExactOutput(
        IERC20 inputToken,
+       uint256 maxInputAmount,
.
.
.
        inputAmount = getInputAmountBasedOnOutput(
            outputAmount,
            inputReserves,
            outputReserves
        );
+       if (inputAmount > maxInputAmount) {
+           revert();
+       }

        _swap(inputToken, inputAmount, outputToken, outputAmount);
```

HIGH [H-3] `TSwapPool::sellPoolTokens` mismatches input and output tokens causing users to receive the incorrect amount of tokens

Description: The `sellPoolTokens` function is intended to allow users to easily sell pool tokens and receive WETH in exchange. Users indicate how many pool tokens they're willing to sell in the `poolTokenAmount` parameter. However, the function currently miscalculates the swapped amount.

This is due to the fact that the `swapExactOutput` function is called, whereas the `swapExactInput` function is the one that should be called. Because users specify the exact amount of input tokens, not output.

Impact: User will swap the wrong amount of tokens, which is a severe disruption of protocol functionality.

Proof of Concept:

You should test with `test_sellPoolTokens_swapsInputAndOutput()` function in `TSwapPool.t.sol`:

```
    function test_sellPoolTokens_swapsInputAndOutput() public {
        uint256 INPUT_RESERVES = 100e18;
        uint256 OUTPUT_RESERVES = 100e18;

        // Extra mint: because of the bug the user is charged FAR more pool tokens than
        // intended (~20e18). Without this the tx would revert on insufficient balance,
        // which is itself a secondary impact of the bug.
        poolToken.mint(user, 20e18);

        // Seed the pool with liquidity so the swap has reserves to price against.
        vm.startPrank(liquidityProvider);
        weth.approve(address(pool), INPUT_RESERVES);
        poolToken.approve(address(pool), OUTPUT_RESERVES);
        pool.deposit(INPUT_RESERVES, 90e18, OUTPUT_RESERVES, uint64(block.timestamp));
        vm.stopPrank();

        // The user's intent: sell exactly 2 pool tokens.
        uint256 expectedPoolTokenSold = 2e18;

        uint256 startingUserPoolTokenBalance = poolToken.balanceOf(user);
        uint256 startingUserWethBalance = weth.balanceOf(user);

        vm.startPrank(user);
        poolToken.approve(address(pool), type(uint256).max);
```

```
        pool.sellPoolTokens(expectedPoolTokenSold);
        vm.stopPrank();

        uint256 endingUserPoolTokenBalance = poolToken.balanceOf(user);
        uint256 endingUserWethBalance = weth.balanceOf(user);

        // SYMPTOM: the user actually pays more pool tokens than they meant to sell.
        uint256 actualPoolTokenSold = startingUserPoolTokenBalance -
endingUserPoolTokenBalance;
        assertGt(actualPoolTokenSold, expectedPoolTokenSold);

        console.log("Actual Pool Token sold: ", actualPoolTokenSold);
        // 20.469571981249872065 poolToken
        console.log("Token Pool user wants to sold: ", expectedPoolTokenSold);
        // 2.000000000000000000 poolToken

        // ROOT CAUSE: the user receives exactly `expectedPoolTokenSold` worth of WETH,
        // proving the value was used as the OUTPUT (WETH out) instead of the INPUT
        // (pool tokens to sell).
        uint256 userWethReceive = endingUserWethBalance - startingUserWethBalance;

        assert(userWethReceive == expectedPoolTokenSold);
        console.log("User Weth Receive: ", userWethReceive);
        // 2.000000000000000000 WETH
        console.log("Token Pool user wants to sold: ", expectedPoolTokenSold);
        // 2.000000000000000000 poolToken
    }
```

Recommended Mitigation:

consider changing the implementation to use `swapExactInput` instead of `swapExactOutput`. Note that this would also require changing the `sellPoolToken` function to accept a new parameter (ie `minWethToReceive` to be passed to `swapExactInput`).

```
    function sellPoolTokens(
        uint256 poolTokenAmount,
+       uint256 minWethToReceive,
    ) external returns (uint256 wethAmount) {
        return
-           swapExactOutput(
-               i_poolToken,
-               i_wethToken,
-               poolTokenAmount,
-               uint64(block.timestamp)
-           );
+           swapExactInput(
+               i_poolToken,
+               poolTokenAmount,
+               i_wethToken,
+               minWethToReceive
+               uint64(block.timestamp)
+           );
    }
```

Additionally, it might be wise to add a deadline to the function, as there is currently no deadline.

HIGH [H-4] In `TSwapPool::_swap` the extra tokens given to users after every `swapCount` breaks the protocol invariant of `x * y = k`

Description: The protocol follow a strict invariant of `x * y = k`. Where: - `x`: The balance of the pool token - `y`: The balance of WETH - `k`: the constant product of the two balances

This means, that whenever the balances change in the protocol, the ratio between the two amount should remain constant, hence the `k`. However, this is broken due to the extra incentive in the `_swap` function. Meaning that over time the protocol funds will be drained.

The follow block of code is responsible for the issue.

```
        swap_count++;
        if (swap_count >= SWAP_COUNT_MAX) {
            swap_count = 0;
            outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000);
        }
```

Impact: A user could maliciously drain the protocol of funds by doing a lot of swaps and collecting the extra incentive given out by the protocol.

Most simply put, the protocol's core invariant is broken.

Proof of Concept: 1. A user swaps 10 times and collects the extra incentive of `1_000_000_000_000_000_000` tokens 2. That user continues the swap until all the protocol funds are drained

Place the following into `TSwapPool.t.sol`.

```
    function testInvariantBroken() public {
        vm.startPrank(liquidityProvider);
        weth.approve(address(pool), 100e18);
        poolToken.approve(address(pool), 100e18);
        pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
        vm.stopPrank();

        uint256 outputWeth = 1e17;

        vm.startPrank(user);
        poolToken.approve(address(pool), type(uint256).max);
        poolToken.mint(user, 100e18);
        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));

        int256 startingY = int256(weth.balanceOf(address(pool)));
        int256 expectedDeltaY = int256(-1) * int256(outputWeth);

        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
        vm.stopPrank();

        uint256 endingY = weth.balanceOf(address(pool));
        int256 actualDeltaY = int256(endingY) - int256(startingY);
```

```
        assertEq(actualDeltaY, expectedDeltaY);
    }
```

Recommended Mitigation: Remove the extra incentive mechanism. If you want to keep this in, you should account for the change in the `x * y = k` protocol invariant. Or, you should set aside tokens in the same way you do with fees.

```
-       swap_count++;
-       if (swap_count >= SWAP_COUNT_MAX) {
-           swap_count = 0;
-           outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000);
-       }
```

Medium

MEDIUM [M-1] `TSwapPool::deposit` is missing deadline check causing transactions to complete even after the deadline

Description: The `deposit` function accepts a deadline parameter, which according to the documentation is "The deadline for the transaction to be completed by". However, this parameter is never used. As a consequence, operations that add liquidity to the pool might be executed at unexpected times, in market conditions where the deposit rate is unfavorable.

Impact: Transactions could be sent when market conditions are unfavorable to deposit, even when adding a deadline parameter.

Proof of Concept: The `deadline` parameter is unused.

Contests of the compiler:

```
forge build

Warning (5667): Unused function parameter. Remove or comment out the variable name to
silence this warning.
   --> src/TSwapPool.sol:124:9:
    |
124 |         uint64 deadline
    |         ^^^^^^^^^^^^^^^
```

Recommended Mitigation: Consider making the following change to the function.

```
    function deposit(
        uint256 wethToDeposit,
        uint256 minimumLiquidityTokensToMint,
        uint256 maximumPoolTokensToDeposit,
        uint64 deadline
    )
        external
+       revertIfDeadlinePassed(deadline)
        revertIfZero(wethToDeposit)
        returns (uint256 liquidityTokensToMint)
    {
```

Low

LOW [L-1] `TSwapPool::LiquidityAdded` event has parameters out of order

Description: When the `LiquidityAdded` event is emitted in the `_addLiquidityMintAndTransfer` function, it logs values in an incorrect order. The `poolTokenToDeposit` value should go in the third parameter position, whereas the `wethToDeposit` value should go second.

Impact: Event emission is incorrect, leading to off-chain function potentially malfunctioning.

Recommended Mitigation:

```
-    emit LiquidityAdded(msg.sender, poolTokensToDeposit, wethToDeposit);
+    emit LiquidityAdded(msg.sender, wethToDeposit, poolTokensToDeposit);
```

LOW [L-2] Default value returned by `TSwapPool::swapExactInput` results in incorrect return value given

Description: The `swapExactInput` function is expected to return the actual amount of token bought by the caller. However, while it declared the name return value `output`, it is never assigned a value, nor uses an explict return statement.

Impact: The return value will always be 0, giving incorrect information to the caller.

Proof of Concept:

You should test with `test_swapExactInput_returnsZeroOutput()` function in `TSwapPool.t.sol`:

```
    function test_swapExactInput_returnsZeroOutput() public {
        vm.startPrank(liquidityProvider);
        weth.approve(address(pool), 100e18);
        poolToken.approve(address(pool), 100e18);
        pool.deposit(100e18, 90e18, 100e18, uint64(block.timestamp));
        vm.stopPrank();

        vm.startPrank(user);
        uint256 startingPoolTokenBalanceUser = poolToken.balanceOf(user);
        weth.approve(address(pool), 5e18);
        uint256 output = pool.swapExactInput(weth, 5e18, poolToken, 3e18,
uint64(block.timestamp));

        uint256 endingPoolTokenBalanceUser = poolToken.balanceOf(user);
        console.log("Starting Pool Token Balance User: ",
startingPoolTokenBalanceUser);
        console.log("Ending Pool Token Balance User: ", endingPoolTokenBalanceUser);

        assert(output == 0);
        assert(endingPoolTokenBalanceUser > startingPoolTokenBalanceUser);
        vm.stopPrank();
    }
```

Recommended Mitigation:

You should change the code to something like this:

```
    {
        uint256 inputReserves = inputToken.balanceOf(address(this));
        uint256 outputReserves = outputToken.balanceOf(address(this));

-       uint256 outputAmount = getOutputAmountBasedOnInput(
```

```
+       output = getOutputAmountBasedOnInput(
            inputAmount,
            inputReserves,
            outputReserves
        );

-       if (outputAmount < minOutputAmount) {
-           revert TSwapPool__OutputTooLow(outputAmount, minOutputAmount);
-       }
+       if (output < minOutputAmount) {
+           revert TSwapPool__OutputTooLow(output, minOutputAmount);
+       }


-       _swap(inputToken, inputAmount, outputToken, outputAmount);
+       _swap(inputToken, inputAmount, outputToken, output);
    }
```

Informational

INFO [I-1] `PoolFactory::PoolFactory__PoolDoesNotExist` is not used and should be removed

```
-   error PoolFactory__PoolDoesNotExist(address tokenAddress);
```

INFO [I-2] Lacking zero address check in `PoolFactory.sol` constructor

```
    constructor(address wethToken) {
+       if (wethToken == address(0)) {
+           revert();
+       }
        i_wethToken = wethToken;
    }
```

INFO [I-3] `PoolFactory::createPool` should use `.symbol()` instead of `.name()` at `PoolFactory::liquidityTokenSymbol` variable

```
-   string memory liquidityTokenSymbol = string.concat("ts",
IERC20(tokenAddress).name());
+   string memory liquidityTokenSymbol = string.concat("ts",
IERC20(tokenAddress).symbol());
```

INFO [I-4] `TSwapPool::Swap` event should be have 3 indexed because there are more than 3 parameters

`tokenIn` dan `tokenOut` should be indexed to enable efficient off-chain filtering by token address. Amounts do not benefit from indexing due to Solidity's inability to perform range queries on topics.

```
    event Swap(
        address indexed swapper,
-       IERC20 tokenIn,
+       IERC20 indexed tokenIn,
        uint256 amountTokenIn,
-       IERC20 tokenOut,
```

```
+       IERC20 tokenOut,
        uint256 amountTokenOut
    );
```

INFO [I-5] Lacking zero address check in `TSwapPool` constructor

```
    constructor(
        address poolToken,
        address wethToken,
        string memory liquidityTokenName,
        string memory liquidityTokenSymbol
    ) ERC20(liquidityTokenName, liquidityTokenSymbol) {
+       if (wethToken == address(0)) {
+           revert();
+       }
+       if (poolToken == address(0)) {
+           revert();
+       }
        i_wethToken = IERC20(wethToken);
        i_poolToken = IERC20(poolToken);
    }
```

Gas

No gas optimizations reported.